

---

# Introduction

Rural communities depend on short and uncertain monsoon seasons, while surface runoff is lost when there are no suitably located storage structures. Farm ponds are an effective rainwater-harvesting intervention: they slow runoff, recharge local soil, and provide water for livestock and irrigation. This assignment delivers JalaSetu, a geospatial-assisted web application that turns contour lines into a terrain model, recommends a pond location, estimates its contributing catchment and expected collected water, and explains the result on an interactive map. The report covers requirements, architecture, algorithms, implementation, CSD design choices, evaluation, and limitations.

## Motivation

Manual pond siting requires interpreting many contour lines, spotting depressions, estimating upstream drainage, and comparing earthwork options. This is slow and error-prone when maps are coarse or when village staff do not have GIS expertise. JalaSetu makes the first-pass assessment repeatable: one contour upload and an optional map-selected land area produce ranked sites, transparent metrics, and boundaries that can be checked in the field. It is a decision-support tool, not a replacement for a surveyor or land records.

## Scope of the Project

The system parses KML/KMZ contours, builds an adaptive local-UTM DEM, computes slope/depression/TWI suitability, ranks candidate pond footprints, allows the user to select a 250 m radius land area on the map, prioritizes sites in that area, delineates its upstream catchment, estimates SCS-CN storm runoff and pond storage, and overlays the selected area, recommended pond, alternatives, and catchment. It does not verify ownership, obtain parcel boundaries, query a live historical rainfall service, perform structural embankment design, guarantee infiltration, or replace a detailed hydrological and geotechnical survey.

## Problem Statement and Requirements

The functional requirements are implemented as follows.

Requirement Implemented in ----- Satellite imagery display  
frontend/src/components/MapView.jsx Esri imagery layer Contour map visualization KML/KMZ parser, DEM pipeline, and Leaflet results map Available-land identification Map click selection with 250 m radius in MapView.jsx Catchment area estimation app/services/catchment\_delineation.py Historical rainfall query Not implemented; design rainfall is an explicit user parameter Runoff volume estimation SCS-CN runoff calculation in catchment\_delineation.py Pond depth / storage recommendation pond\_site.py stage-storage curve and design depth Combined overlay / results view ResultsLayer.jsx, site and catchment panels

: Functional requirements and where they are implemented

## Non-Functional Requirements

The target is a normal analysis response in under 30 seconds for the provided contour dataset and a responsive map interaction below 100 ms. The frontend must build as a static Vite bundle and work in current Chrome, Firefox, Safari, and Edge; the backend is stateless and can run with multiple Uvicorn workers. A deployment target of four API workers and approximately 20 concurrent analyses is reasonable for this assignment, with uploaded files bounded by deployment configuration. GeoJSON is kept to the computed boundaries rather than returning raster arrays. Inputs are validated by FastAPI, temporary uploads are removed in a finally block, and production deployments should replace permissive development CORS and add authentication and rate limiting.

## System Architecture and High-Level Design

The browser provides the contour file and selected map centre/radius to a FastAPI service. The route offloads the CPU-heavy analysis to a worker thread, keeping the event loop available for health checks and concurrent requests. The service runs the parser, DEM builder, terrain analysis, pond siting, and catchment stages, then returns one JSON contract containing metrics and GeoJSON. The present implementation has no persistent database or external elevation/rainfall dependency: the uploaded contours are the source data and the design rainfall/CN are explicit model inputs.

```
React + React-Leaflet map
  | KML/KMZ + selected centre/radius
  v
FastAPI /analyzeContour
  | parser -> DEM -> terrain suitability -> pond ranking
  | -> D8/pysheds catchment -> SCS-CN runoff
  v
CatchmentResponse JSON + GeoJSON overlays
  |
  v
```

## Technology Stack

The backend uses Python 3.9, FastAPI, Pydantic, GeoPandas, Shapely, Rasterio, SciPy, NumPy, and optional pysheds. The frontend uses React 18, Vite, Axios, Leaflet 1.9, and React-Leaflet. GeoJSON is the interchange format. No database or external rainfall/elevation API is queried in the current offline-first version; this avoids rate-limit and availability failures for a contour-only assignment and makes the demo reproducible. Docker/Uvicorn deployment files are included for reproducible hosting.

## Methodology

The pipeline is a linear sequence over an adaptive raster. It releases large intermediate arrays between stages and returns a bounded number of candidate sites (`max_candidate_sites`, default 5), which limits response size and memory pressure.

## Terrain and Elevation Analysis

`kml_parser.py` extracts line geometries and elevation attributes from KML or KMZ. `dem_builder.py` samples contour vertices/segments, reprojects the data to a local UTM CRS, and interpolates a regular DEM using linear interpolation with a nearest-neighbour fallback outside the convex hull. Resolution is data-driven unless overridden (the supplied dataset is processed at metre-scale resolution). Terrain analysis derives gradient slope, local depression/TPI measures, and topographic wetness index. A weighted suitability score combines slope, depression, and TWI; only contiguous regions above the threshold and minimum area are passed to pond siting. This uses geospatial/numerical raster processing rather than OpenCV because the input is vector contour data.

## Catchment Area Delineation

The recommended pond centroid is snapped to a high-accumulation cell within the configured radius. When available, pysheds performs flow direction and accumulation; the native fallback uses D8 neighbour routing and upstream tracing. The resulting mask is polygonized and transformed to WGS84 for the response. Catchment area is computed in projected metres, and mean slope/elevation relief are calculated over the mask. The model assumes a static DEM, one pour point, and no culverts, roads, soil infiltration variation, or engineered diversions.

## Rainfall Data Integration

No live historical rainfall API is queried. Instead, the user-configurable 24-hour design rainfall defaults to 100 mm and the agricultural runoff curve number defaults to 75. The SCS-CN equation converts rainfall excess and catchment area to `estimated_runoff_volume_m3`. This is deliberately explicit in the UI/API so a village engineer can replace the defaults with an approved local design storm.

## Implementation

### Backend and API Design

| Method / path              | Purpose                                                                                                    |
|----------------------------|------------------------------------------------------------------------------------------------------------|
| GET /health                | Service liveness check.                                                                                    |
| POST /analyzeContour       | Upload KML/KMZ, optionally include selected latitude, longitude and radius, and run the complete pipeline. |
| POST /findCatchment        | Backward-compatible alias for the same analysis.                                                           |
| GET /analyzeContour/schema | Return response schema and default parameters.                                                             |

The API has no authentication in this academic prototype. FastAPI validates file type, XML/ZIP integrity, numeric ranges, and slope relationships. Pipeline failures are returned as explicit status/message/processing-notes responses; HTTP 400/422/500 are used for invalid upload, unprocessable input, and unexpected server failures respectively.

### Frontend and Visualization

The React client uses Leaflet base layers for Street, Satellite, and Hybrid views. Clicking the map places a centre marker and a dashed 250 m selection circle. Clicking Analyze sends that area to the backend. The returned recommended pond footprint is amber, alternatives are slate, the upstream catchment is translucent blue, and each polygon has a metrics popup. Panels show recommended latitude/longitude, footprint, slope, design storage, catchment hectares, SCS runoff volume, rainfall/CN assumptions, and feasibility notes. The production bundle was verified with `npm run build`.

## Database and Storage

The current application is intentionally stateless: contour files are stored only in a temporary server file during a request and deleted after completion; computed results are returned as JSON/GeoJSON. This is suitable for a demonstrator and avoids retaining land data. A production extension would use PostGIS for villages, parcels, cached DEM metadata, rainfall observations, and versioned analysis results, with spatial indexes on geometry and a request/job identifier.

## CSD Themes and Topics Applied in the Project

| CSD Theme/Topic                        | Where used in the project                                           | Justification / design rationale                                                                      |
|----------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| API Design (REST)                      | FastAPI /health, /analyzeContour, /findCatchment                    | A typed JSON contract keeps browser and pipeline independently testable.                              |
| Load Balancing                         | Uvicorn worker deployment configuration                             | Stateless requests can be distributed across workers for concurrent village queries.                  |
| Caching                                | Not implemented in the prototype                                    | Avoids stale terrain results; production can cache immutable contour hashes and rainfall responses.   |
| Database Indexing / Query Optimization | Bounded candidate count and projected-area calculations             | Limits response size; PostGIS indexes are a production extension, not claimed here.                   |
| Concurrency / Asynchronous Processing  | Async FastAPI route with <code>run_in_threadpool</code>             | CPU-heavy raster work does not block the event loop, so health checks and other requests can proceed. |
| Microservices vs. Monolith             | Modular monolith in <code>app/services/</code>                      | Shared in-memory DEM state makes one request cheaper and simpler than network hops.                   |
| Design Patterns                        | Pipeline orchestration, service classes, fallback strategy          | Each stage has one responsibility and hydrology degrades explicitly to native D8.                     |
| Containerization / Deployment          | Dockerfiles and Render/Uvicorn configuration                        | Reproduces Python and frontend deployment environments.                                               |
| Error Handling and Resilience          | Input validation, status values, processing notes, cleanup          | Invalid files and missing optional hydrology support surface actionable results.                      |
| Algorithms and Complexity              | Raster gradients, connected components, D8 upstream traversal       | Work scales with raster cells; candidate and response counts are capped.                              |
| Testing Strategy                       | Unit, API, end-to-end, and distributed tests in <code>tests/</code> | Covers parser, DEM, terrain, siting, routes, and complete response shape.                             |
| Version Control / CI-CD                | Git repository with separated app, frontend, distributed, and tests | Keeps deployable surfaces and regression tests reviewable.                                            |

The most significant choices are the modular pipeline and explicit fallbacks. Terrain and hydrology are computationally different concerns, so their services can be optimized independently while the orchestration keeps one request easy to reason about. The selected-area feature is deliberately a bounded circle rather than an unconstrained polygon drawing tool: it is fast, requires no extra drawing dependency, and prevents huge requests while still letting a user target a parcel neighbourhood.

## Results and Evaluation

For the supplied `contours_1m.kml`, the representative run produced a successful response with a recommended pond location and ranked alternative sites. Exact values are data-dependent and are returned by the API rather than hard-coded: the reportable fields are `recommended_site.latitude`, `recommended_site.longitude`, `recommended_site.area_m2`, `recommended_site.storage_capacity_m3`, `catchment.area_hectares`, and `catchment.estimated_runoff_volume_m3`. The map makes all three requested outputs inspectable: the pond location/footprint, catchment boundary/area, and expected water volume (storage and storm runoff) are shown in panels and

---

popups. The selected-area response also records centre, radius, and area.

*Example results overlay for a selected village.*

## Performance

The targeted backend regression suite completed 11 tests in 12.56 seconds, including API route and end-to-end coverage. The Vite production build completed successfully in approximately 1 second and generated a 406.5 kB JavaScript bundle (124.7 kB gzip) plus 42.9 kB CSS (11.8 kB gzip). These figures include test/build startup overhead and are not a production load benchmark. Raster cost is approximately linear in DEM cell count; for stress scaling, use adaptive resolution, worker processes, upload limits, request timeouts, and a job queue for very large files.

## Discussion and Limitations

The implementation is strong as a reproducible first-pass terrain screening workflow: it uses projected geometry for area, emits standard GeoJSON, exposes assumptions, and has a tested fallback when pysheds is not available. Accuracy remains limited by contour spacing/interpolation, unknown soil and land-cover parameters, the single design storm, and the absence of parcel ownership and field survey data. The selected map circle is a planning area, not a legal parcel. A production release should add authenticated users, PostGIS, rainfall history with provenance, DEM quality checks, background jobs, rate limiting, observability, and field validation.

## AI Tool Usage Declaration

AI tools, including GitHub Copilot, were used for repository inspection, implementation assistance, debugging, frontend integration, and report editing. The team reviewed the generated changes, understood the pipeline and API contract, and validated the final implementation with the frontend production build and targeted backend tests. AI assistance did not replace engineering judgement, field validation, or responsibility for the submitted design and results.

## Source Code and Repository

The source repository is <https://github.com/Lazypilottt/Jalasetu>. The root contains the FastAPI application in `app/`, the React/Vite client in `frontend/`, optional distributed-stage demonstrations in `distributed/`, regression tests in `tests/`, deployment files, the supplied `contours_1m.kml`, and this report template filled with the implemented system details.

## Appendix: Reproducibility and Operations

1. Start the backend with `uvicorn app.main:app --host 0.0.0.0 --port 8000`.
2. Start the frontend with `cd frontend && npm run dev`.
3. Set `VITE_API_BASE_URL` when the API is not at `http://127.0.0.1:8000`.
4. Upload `contours_1m.kml`, click the target area on the map, and press

Analyze. Inspect the amber pond, blue catchment, green selected area, storage capacity, runoff volume, and processing notes.

5. Run backend regression checks with

`python -m pytest tests/` and build checks with `cd frontend && npm run build`.